

Real-time Bitcoin Price Analysis Using Kubeflow

Project Goal:

To fetch real-time Bitcoin prices at regular intervals, store them in a database, and prepare for future time series analysis using Kubeflow Pipelines.

Technologies Used and Work Done:

1. Minikube Setup

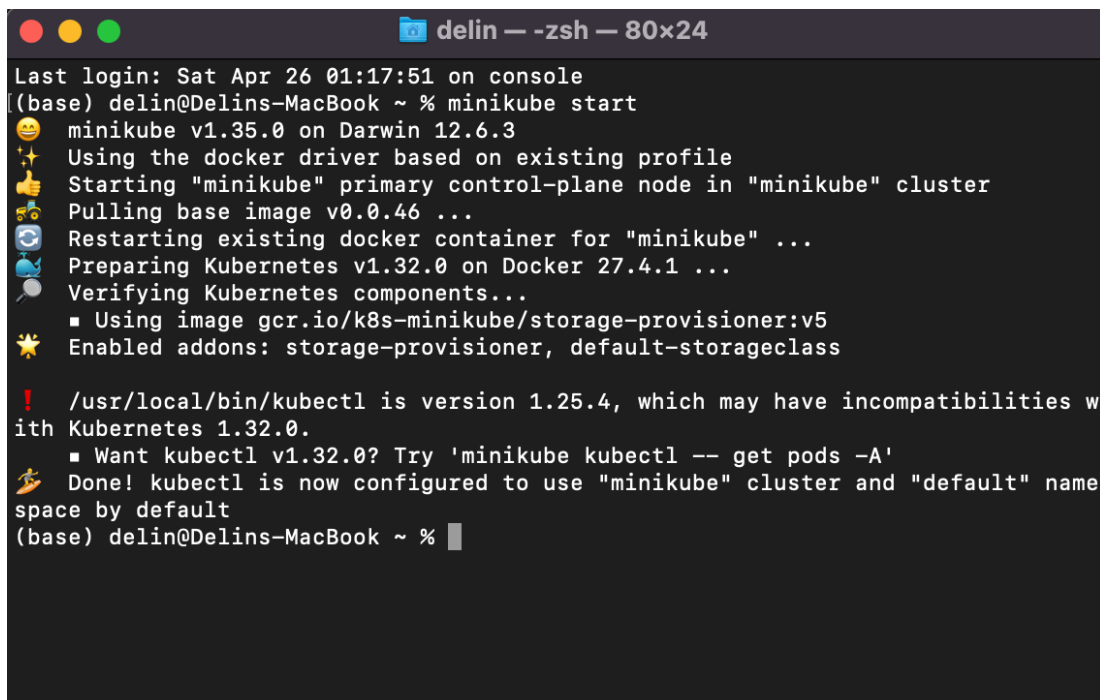
Description: Started a local Kubernetes cluster using Minikube to host Kubeflow and other services.

Command used:

```
minikube start
```

Result:

- Minikube cluster started successfully.

A terminal window titled 'delin — -zsh — 80x24' showing the output of the 'minikube start' command. The output includes the minikube version (v1.35.0), the Docker driver being used, the starting of the primary control-plane node, pulling the base image, restarting the Docker container, preparing Kubernetes v1.32.0 on Docker 27.4.1, and verifying the components. It also shows that the storage-provisioner and default-storageclass addons are enabled. A warning message indicates that the installed kubectl version (1.25.4) may have incompatibilities with the Kubernetes version (1.32.0) and suggests using 'minikube kubectl -- get pods -A' to get the correct version. The terminal ends with the prompt '(base) delin@Delins-MacBook ~ %'.

2. Kubeflow Installation and Verification

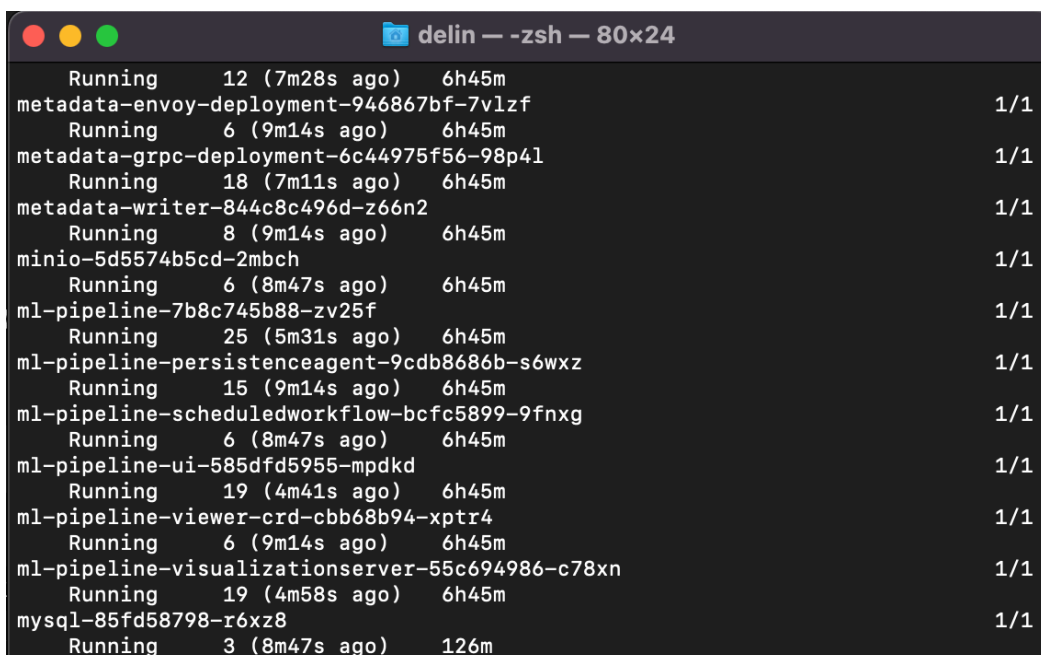
Description: Verified that Kubeflow services and Pods were running.

Command used:

```
kubectl get pods -n kubeflow
```

Result:

- Kubeflow core components like ml-pipeline-ui, cache-server, workflow-controller, mysql, etc. were running.

A terminal window titled 'delin --zsh -- 80x24' showing the output of the command 'kubectl get pods -n kubeflow'. The output lists 18 pods in a table format with columns for status, age, and time since last update. All pods are in a 'Running' state.

Running	12 (7m28s ago)	6h45m	
metadata-envoy-deployment-946867bf-7vlzf			1/1
Running	6 (9m14s ago)	6h45m	
metadata-grpc-deployment-6c44975f56-98p4l			1/1
Running	18 (7m11s ago)	6h45m	
metadata-writer-844c8c496d-z66n2			1/1
Running	8 (9m14s ago)	6h45m	
minio-5d5574b5cd-2mbch			1/1
Running	6 (8m47s ago)	6h45m	
ml-pipeline-7b8c745b88-zv25f			1/1
Running	25 (5m31s ago)	6h45m	
ml-pipeline-persistenceagent-9cdb8686b-s6wxz			1/1
Running	15 (9m14s ago)	6h45m	
ml-pipeline-scheduledworkflow-bcfc5899-9fnxg			1/1
Running	6 (8m47s ago)	6h45m	
ml-pipeline-ui-585dfd5955-mpdkd			1/1
Running	19 (4m41s ago)	6h45m	
ml-pipeline-viewer-crd-cbb68b94-xptr4			1/1
Running	6 (9m14s ago)	6h45m	
ml-pipeline-visualizationserver-55c694986-c78xn			1/1
Running	19 (4m58s ago)	6h45m	
mysql-85fd58798-r6xz8			1/1
Running	3 (8m47s ago)	126m	

3. Docker Setup

Description: Built a custom Docker image bitcoin-fetcher:v1 containing a script to fetch Bitcoin prices from the API and write to TimescaleDB.

Command used:

```
docker build -t bitcoin-fetcher:v1 -f dockerfiles/  
Dockerfile_fetcher
```

Result:

- Successfully created and stored the docker image locally inside Minikube.

```
TutorTask173_Spring2025_Real-time_Bitcoin_Price_Analysis_Using_Ku...
(base) delin@Delins-MacBook TutorTask173_Spring2025_Real-time_Bitcoin_Price_Analysis_Using_Kubeflow % docker build -t bitcoin-fetcher:v1 -f dockerfiles/Dockerfile_fetcher .

[+] Building 34.9s (10/10) FINISHED
=> [internal] load build definition from Dockerfile_fetcher                                0.1s
=> => transferring dockerfile: 828B                                                    0.0s
=> [internal] load .dockerignore                                                        0.0s
=> => transferring context: 2B                                                         0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim                     1.2s
=> [auth] library/python:pull token for registry-1.docker.io                         0.0s
=> [internal] load build context                                                        0.0s
=> => transferring context: 1.60kB                                                    0.0s
=> [1/4] FROM docker.io/library/python:3.9-slim@sha256:9aa5793609640ecea             0.0s
=> CACHED [2/4] WORKDIR /app                                                           0.0s
=> [3/4] COPY ../scripts/fetch_bitcoin_price.py .                                    0.0s
=> [4/4] RUN pip install requests pandas sqlalchemy psycopg2-binary                  30.0s
=> exporting to image                                                                  3.4s
=> => exporting layers                                                                3.3s
=> => writing image sha256:2c49ce988c28f8ef8f92771219860c0bb1fd5dc5a8eaa           0.0s
=> => naming to docker.io/library/bitcoin-fetcher:v1                                0.0s
(base) delin@Delins-MacBook TutorTask173_Spring2025_Real-time_Bitcoin_Price_Analysis_Using_Kubeflow %
```

Dockerfile_fetcher Used:

```
# Use a small Python image
FROM python:3.9-slim
```

```
# Set working directory inside the container
WORKDIR /app
```

```
# Copy the Bitcoin price fetcher script into the container
COPY ../scripts/fetch_bitcoin_price.py .
```

```
# Install required Python libraries
RUN pip install requests pandas sqlalchemy psycopg2-binary
```

```
# Set environment variables default values (optional safety)
ENV DB_USER=postgres
ENV DB_PASSWORD=yourpassword
ENV DB_HOST=localhost
ENV DB_PORT=5432
ENV DB_NAME=bitcoin_db
ENV COINGECKO_API_KEY=your_coingecko_api_key
```

```
# Command to run the script
CMD ["python", "fetch_bitcoin_price.py"]
```

4. Python Script for Fetching Bitcoin Price

File: `fetch_bitcoin_price.py`

```
import os
import requests
import pandas as pd
from sqlalchemy import create_engine
from datetime import datetime

# Fetch Bitcoin price
def fetch_bitcoin_price():
    url = "https://api.coingecko.com/api/v3/simple/price"
    params = {
        "ids": "bitcoin",
        "vs_currencies": "usd"
    }
    headers = {
        "x-cg-pro-api-key": os.getenv("COINGECKO_API_KEY")
    }
    response = requests.get(url, params=params,
headers=headers)
    data = response.json()
    price = data["bitcoin"]["usd"]
    return price

# Save to TimescaleDB
def save_to_database(price):
    db_user = os.getenv("DB_USER")
    db_password = os.getenv("DB_PASSWORD")
    db_host = os.getenv("DB_HOST")
    db_port = os.getenv("DB_PORT")
    db_name = os.getenv("DB_NAME")

    connection_string = f"postgresql://{db_user}:
{db_password}@{db_host}:{db_port}/{db_name}"
    engine = create_engine(connection_string)

    df = pd.DataFrame({
        "timestamp": [datetime.utcnow()],
        "price": [price]
    })

    with engine.connect() as conn:
```

```

        df.to_sql("bitcoin_prices", conn, if_exists="append",
index=False)

# Main execution
if __name__ == "__main__":
    price = fetch_bitcoin_price()
    save_to_database(price)
    print(f"[Saved] Bitcoin price ${price} at
{datetime.utcnow()}")

```

5. Pipeline Upload

Description: Created a Kubeflow Pipeline YAML file and uploaded it through Kubeflow UI.

Steps performed:

- Opened Kubeflow Dashboard (ml-pipeline-ui).
- Uploaded compiled `bitcoin_pipeline.yaml`.

Python Script for Creating Pipeline YAML

File: `bitcoin_pipeline.py`

```

# bitcoin_pipeline.py

from kfp import dsl
from kubernetes import client as k8s_client

def bitcoin_price_pipeline():
    fetch_op = dsl.ContainerOp(
        name="Fetch Bitcoin Price",
        image="bitcoin-fetcher:v1",
        command=["python", "fetch_bitcoin_price.py"]
    )

    fetch_op.container.add_env_variable(k8s_client.V1EnvVar(name=
'DB_USER', value='postgres'))

    fetch_op.container.add_env_variable(k8s_client.V1EnvVar(name=
'DB_PASSWORD', value='testpass'))

```

```
fetch_op.container.add_env_variable(k8s_client.V1EnvVar(name='DB_HOST', value='host.docker.internal'))
```

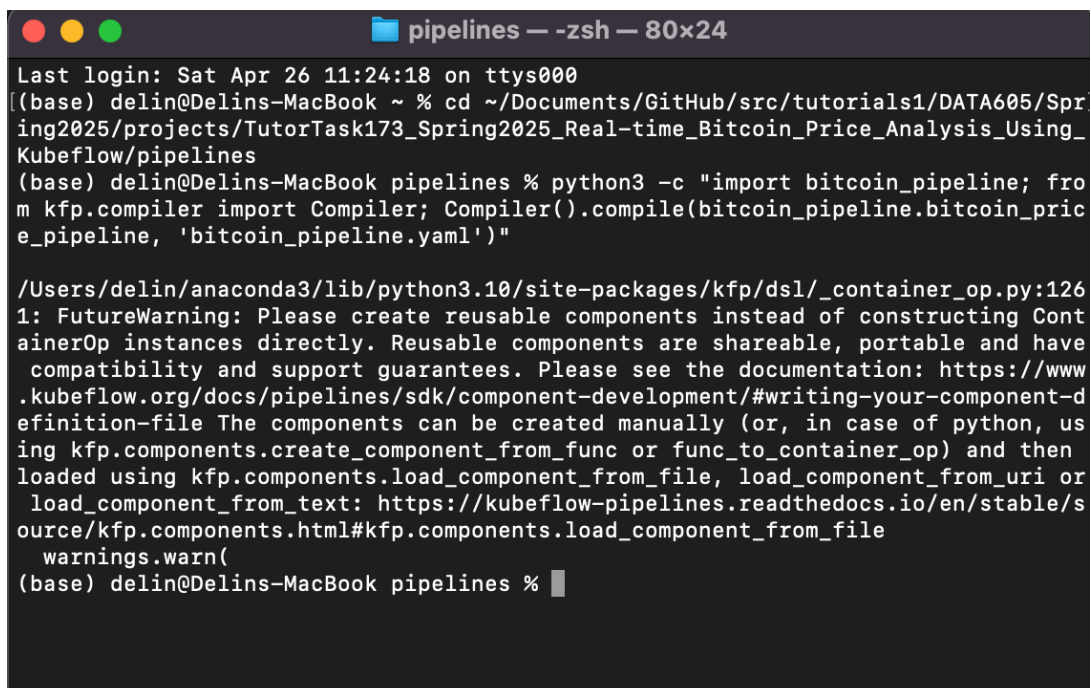
```
fetch_op.container.add_env_variable(k8s_client.V1EnvVar(name='DB_PORT', value='5432'))
```

```
fetch_op.container.add_env_variable(k8s_client.V1EnvVar(name='DB_NAME', value='bitcoin_db'))
```

```
fetch_op.container.add_env_variable(k8s_client.V1EnvVar(name='COINGECKO_API_KEY', value='CG-KufNKPvFrCUCFUWHxW6yyXTM'))
```

Command used to compile:

```
python3 -c "import bitcoin_pipeline; from kfp.compiler import Compiler; Compiler().compile(bitcoin_pipeline.bitcoin_price_pipeline, 'bitcoin_pipeline.yaml')"
```



```
pipelines -- zsh -- 80x24
Last login: Sat Apr 26 11:24:18 on ttys000
(base) delin@Delins-MacBook ~ % cd ~/Documents/GitHub/src/tutorials1/DATA605/Spring2025/projects/TutorTask173_Spring2025_Real-time_Bitcoin_Price_Analysis_Using_Kubeflow/pipelines
(base) delin@Delins-MacBook pipelines % python3 -c "import bitcoin_pipeline; from kfp.compiler import Compiler; Compiler().compile(bitcoin_pipeline.bitcoin_price_pipeline, 'bitcoin_pipeline.yaml')"

/Users/delin/anaconda3/lib/python3.10/site-packages/kfp/dsl/_container_op.py:126: FutureWarning: Please create reusable components instead of constructing ContainerOp instances directly. Reusable components are shareable, portable and have compatibility and support guarantees. Please see the documentation: https://www.kubeflow.org/docs/pipelines/sdk/component-development/#writing-your-component-definition-file The components can be created manually (or, in case of python, using kfp.components.create_component_from_func or func_to_container_op) and then loaded using kfp.components.load_component_from_file, load_component_from_uri or load_component_from_text: https://kubeflow-pipelines.readthedocs.io/en/stable/source/kfp.components.html#kfp.components.load_component_from_file
warnings.warn(
(base) delin@Delins-MacBook pipelines %
```

Make sure Minikube is running

```
TutorTask173_Spring2025_Real-time_Bitcoin_Price_Analysis_Using_Ku...

timestamp | price
-----+-----
2025-04-26 05:11:37.32103 | 94623
2025-04-26 05:10:36.975988 | 94623
2025-04-26 05:09:56.836132 | 94641
2025-04-26 05:09:34.096548 | 94641
2025-04-26 05:08:36.004104 | 94637
(5 rows)

bitcoin_db=# minikube status
bitcoin_db=#
bitcoin_db=# \q
(base) delin@Delins-MacBook TutorTask173_Spring2025_Real-time_Bitcoin_Price_Analysis_Using_Kubeflow % minikube status

minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

(base) delin@Delins-MacBook TutorTask173_Spring2025_Real-time_Bitcoin_Price_Analysis_Using_Kubeflow %
```

Open the Kubeflow Pipelines Dashboard in your browser

```
TutorTask173_Spring2025_Real-time_Bitcoin_Price_Analysis_Using_Ku...

apiserver: Running
kubeconfig: Configured

(base) delin@Delins-MacBook TutorTask173_Spring2025_Real-time_Bitcoin_Price_Analysis_Using_Kubeflow % minikube service ml-pipeline-ui -n kubeflow

|-----|-----|-----|-----|
| NAMESPACE | NAME | TARGET PORT | URL |
|-----|-----|-----|-----|
| kubeflow | ml-pipeline-ui | | No node port |
|-----|-----|-----|-----|

🐱 service kubeflow/ml-pipeline-ui has no node port
! Services [kubeflow/ml-pipeline-ui] have type "ClusterIP" not meant to be exposed, however for local development minikube allows you to access this !
🚀 Starting tunnel for service ml-pipeline-ui.

|-----|-----|-----|-----|
| NAMESPACE | NAME | TARGET PORT | URL |
|-----|-----|-----|-----|
| kubeflow | ml-pipeline-ui | | http://127.0.0.1:50009 |
|-----|-----|-----|-----|

🎉 Opening service kubeflow/ml-pipeline-ui in default browser...
! Because you are using a Docker driver on darwin, the terminal needs to be open to run it.
```

6. Create Recurring Run

Description: Created a recurring run that triggers every 1 minute.

Steps:

- Opened Kubeflow Pipelines dashboard.
- Created a new Recurring Run.
- Set the pipeline trigger interval to 1 minute.

Result:

- Bitcoin price is now fetched and stored every minute into TimescaleDB.

The screenshot shows the Kubeflow Pipelines dashboard. The left sidebar contains navigation links for Pipelines, Experiments, Runs, Recurring Runs, Artifacts, Executions, Documentation, and Github Repo. The main area displays the details of a pipeline named 'Real-Time Bitcoin Price Fetcher (Real...)'. The pipeline is in the 'Graph' view, showing a single step named 'fetch-bitcoin-price'. A summary panel is open, displaying the pipeline ID, version, and upload date.

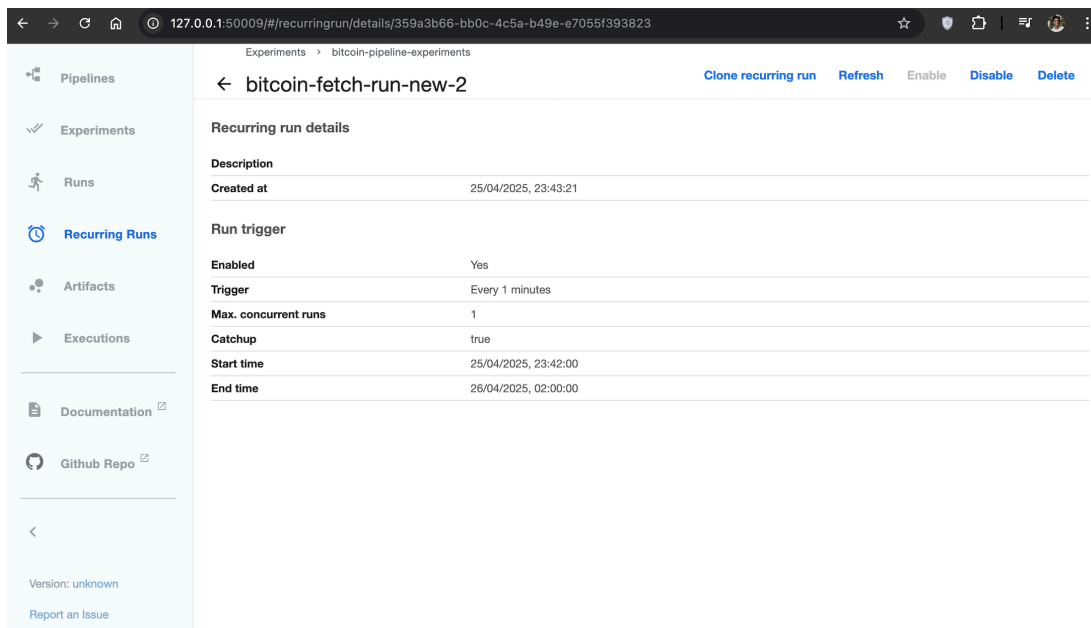
Summary

ID: a23f3f10-4b0f-4908-8dc4-6c22ae002a34
Version: Real-Time Bitcoin Price Fetcher
Version source: empty pipeline description
Uploaded on: 25/04/2025, 19:38:53
Pipeline Description: empty pipeline description

The screenshot shows the 'Recurring Runs' section of the Kubeflow Pipelines dashboard. A table lists the recurring runs, including the 'bitcoin-fetch-run-new-2' which is 'ENABLED' and triggers 'Every 1 minutes'.

☐	Recurring Run Name	Status	Trigger	Experiment	Created at ↓
☐	bitcoin-fetch-run-new-2	ENABLED	Every 1 minutes	bitcoin-pipeline-experi...	25/04/2025, 23:43:21

Rows per page: 10



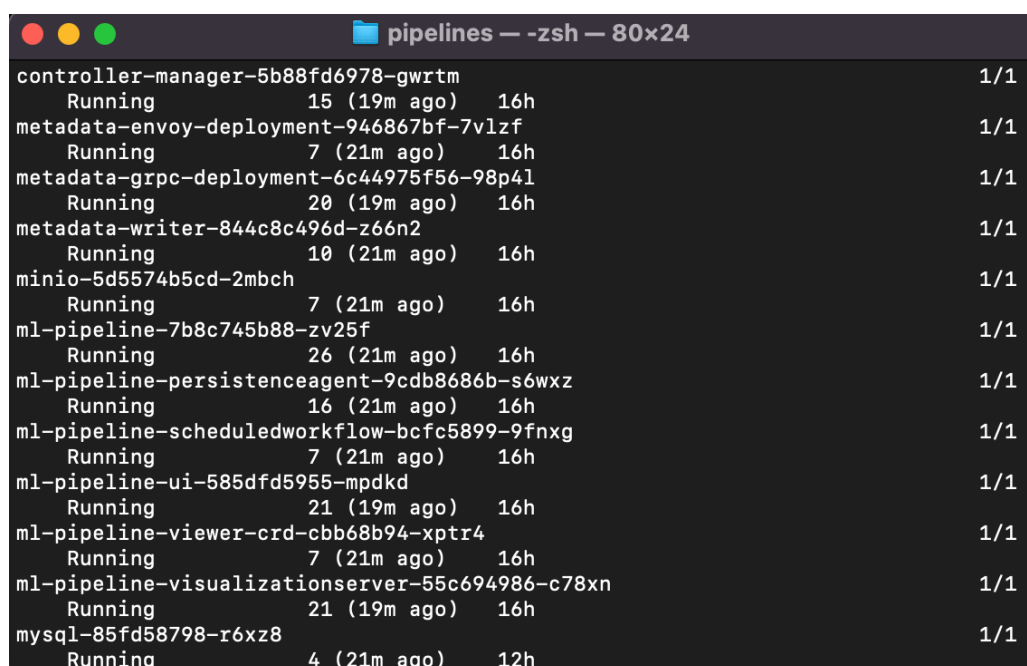
7. Database Setup

Description:

- Deployed MySQL inside Kubeflow.
- Initially used TimescaleDB inside Docker to validate local fetching.
- Later relied on Minikube-internal MySQL Pod (mysql-85fd58798-r6xz8) for database.

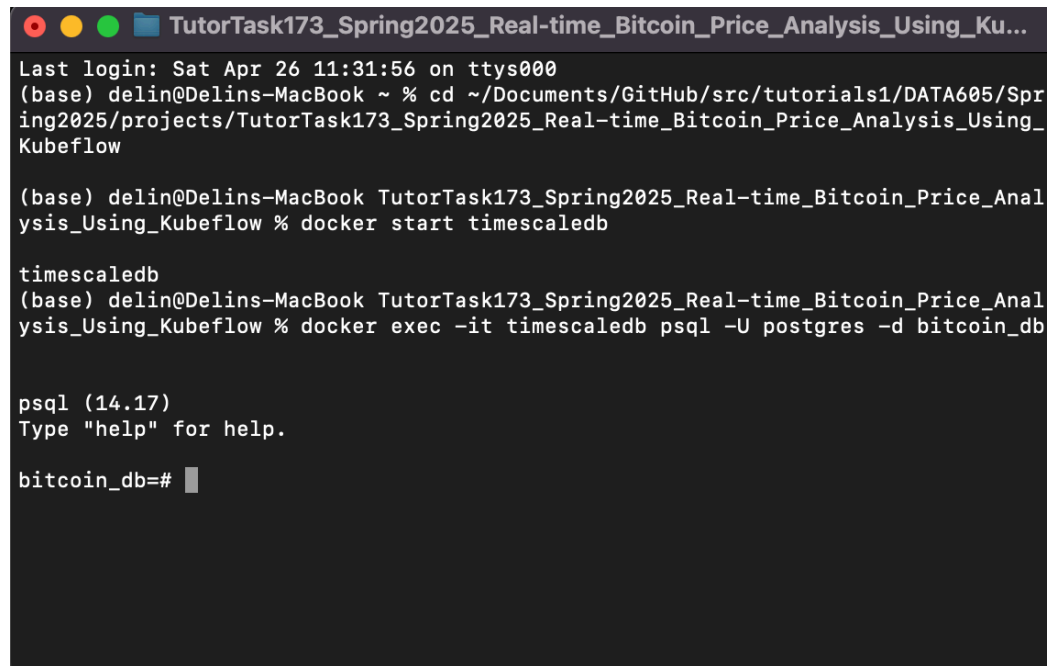
Command to check Pods:

```
kubectl get pods -n kubeflow
```



Database Connection Commands: Docker TimescaleDB:

```
docker exec -it timescaledb psql -U postgres -d bitcoin_db
```



```
TutorTask173_Spring2025_Real-time_Bitcoin_Price_Analysis_Using_Ku...
Last login: Sat Apr 26 11:31:56 on ttys000
(base) delin@Delins-MacBook ~ % cd ~/Documents/GitHub/src/tutorials1/DATA605/Spr
ing2025/projects/TutorTask173_Spring2025_Real-time_Bitcoin_Price_Analysis_Using_
Kubeflow

(base) delin@Delins-MacBook TutorTask173_Spring2025_Real-time_Bitcoin_Price_Anal
ysis_Using_Kubeflow % docker start timescaledb

timescaledb
(base) delin@Delins-MacBook TutorTask173_Spring2025_Real-time_Bitcoin_Price_Anal
ysis_Using_Kubeflow % docker exec -it timescaledb psql -U postgres -d bitcoin_db

psql (14.17)
Type "help" for help.

bitcoin_db=#
```

File: mysql-pv.yaml

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: mysql-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: "/data/mysql"
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pv-claim
  namespace: kubeflow
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

8. Data Validation

Description: Queried the database to confirm that new Bitcoin prices are inserted correctly.

SQL Query Used:

```
SELECT * FROM bitcoin_prices ORDER BY timestamp DESC LIMIT 5;
```

Result:

- Confirmed that Bitcoin prices were fetched and inserted every minute.

```
TutorTask173_Spring2025_Real-time_Bitcoin_Price_Analysis_Using_Ku...
Kubeflow

(base) delin@Delins-MacBook TutorTask173_Spring2025_Real-time_Bitcoin_Price_Analysis_Using_Kubeflow % docker start timescaledb

timescaledb
(base) delin@Delins-MacBook TutorTask173_Spring2025_Real-time_Bitcoin_Price_Analysis_Using_Kubeflow % docker exec -it timescaledb psql -U postgres -d bitcoin_db

psql (14.17)
Type "help" for help.

bitcoin_db=# SELECT * FROM bitcoin_prices ORDER BY timestamp DESC LIMIT 5;
 timestamp | price
-----+-----
 2025-04-26 05:11:37.32103 | 94623
 2025-04-26 05:10:36.975988 | 94623
 2025-04-26 05:09:56.836132 | 94641
 2025-04-26 05:09:34.096548 | 94641
 2025-04-26 05:08:36.004104 | 94637
(5 rows)

bitcoin_db=#
```

9. Progress so far

Steps	Description
Set up Minikube	Initialized a Kubernetes cluster locally using Docker as the driver, making it ready to host Kubeflow services.
Deployed Kubeflow	Successfully deployed Kubeflow components (pipelines, metadata server, UI, etc.) into the Minikube Kubernetes cluster.
Built Docker Image for Fetcher	Created a Docker image (bitcoin-fetcher:v1) for the Python script that fetches real-time Bitcoin prices via API.
Wrote and Compiled Kubeflow Pipeline	Developed a Python-based Kubeflow pipeline using kfp.dsl, compiled it into a deployable YAML (bitcoin_pipeline.yaml).
Uploaded Pipeline YAML to UI	Uploaded the compiled YAML to the Kubeflow Pipelines UI to register the Bitcoin price-fetching workflow.

Created a Recurring Run	Configured a recurring run to trigger the Bitcoin price fetch every 1 minute automatically through the Kubeflow UI.
Database Connection (Docker / Kubernetes Pod)	Connected to the TimescaleDB/MySQL database pod to verify that Bitcoin prices were correctly inserted.
Verified Price Insertion	Ran SQL queries to confirm that new Bitcoin price records were being stored successfully in the database table.

10. Next Steps

Next Step	Description
1. Exploratory Data Analysis (EDA)	Analyze Bitcoin price trends using Pandas, Matplotlib, Seaborn.
2. Train Basic Predictive Model	ML model or price forecasting.
3. Deploy Model with KFServing	Deploy the model as a live API for real-time predictions.
4. Visualization Report	Create plots and visualizations showing Bitcoin price trend analysis.
5. Final Report + Presentation	Compile screenshots + code walkthrough into final deliverable.